

WOXSEN UNIVERSITY

School of Technology



A

PROJECT REPORT

on

IT HELPDESK AND ASSET SUPPORT MANAGEMENT SYSTEM

Submitted in partial fulfillment of the requirements for the degree of

B. Tech. in Artificial Intelligence and Machine Learning

Submitted by:

Md Aali Rahman

Roll No: 25WU0102156

AIML Panthers

Under the guidance of:

Dr. Kiranmayee Adavala

Assistant Professor

School of Technology

CERTIFICATE

This is to certify that the project report entitled "**IT Helpdesk and Asset Support Management System**" submitted by **Md Aali Rahman** (Roll Number: **25WU0102156**), student of **AIML Panthers**, in partial fulfillment of the requirements for the award of the degree of **B. Tech. in Artificial Intelligence and Machine Learning** from **Woxsen University** is a bonafide record of work carried out by the student under my supervision and guidance.

The work embodied in this project report has been carried out by the candidate and has not been submitted elsewhere for the award of any other degree or diploma.

Signature of Mentor

Name: Dr. Kiranmayee Adavala

Designation: Assistant Professor

Department: Computer Science and Engineering, School of Technology

Institution: Woxsen University, Hyderabad

Date: 10 October 2026

DECLARATION OF THE CANDIDATE

I hereby declare that the project work entitled "**IT Helpdesk and Asset Support Management System**" submitted to the **Department of Computer Science and Engineering, School of Technology, Woxsen University**, in partial fulfillment of the requirements for the award of the degree of **B. Tech. in Artificial Intelligence and Machine Learning** is my original work and has been carried out under the guidance of **Dr. Kiranmayee Adavala**.

I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Signature of Student

Name: Md Aali Rahman

Roll Number: 25WU0102156

Section: AIML Panthers

Date: 10 October 2026

ACKNOWLEDGMENT

I would like to express my sincere gratitude to all those who have contributed to the successful completion of this project on **IT Helpdesk and Asset Support Management System**.

First and foremost, I extend my heartfelt thanks to my project guide, **Dr. Kiranmayee Adavala**, Assistant Professor, School of Technology, for her invaluable guidance, continuous support, and constructive feedback throughout the duration of this project. Her expertise in Database Management Systems, relational design, and software architecture has been instrumental in shaping this work.

I am grateful to **Dr. P. Swapna**, Head of the Department of Computer Science and Engineering, and the academic leadership of the School of Technology, Woxsen University, for providing the necessary computational infrastructure, laboratory facilities, and resources required for developing and testing this project.

My sincere thanks to my peers and colleagues of **AIML Panthers** who provided valuable insights, critical suggestions, and collaborative discussions during various phases of schema design and live verification.

Finally, I am deeply grateful to my family for their unwavering encouragement, understanding, and support throughout my academic journey.

Md Aali Rahman

Roll No: 25WU0102156

AIML Panthers

ABSTRACT

This project presents the design, mathematical normalization, physical implementation, and interactive verification of the **IT Helpdesk and Asset Support Management System**. In modern enterprise and university environments, managing technical service requests and computing hardware through informal channels such as spreadsheets and email threads leads to lost tickets, unmonitored technician workloads, untracked warranty expirations, and missing historical audit trails.

The primary objective of this project is to develop and evaluate a centralized, normalized relational database system using **MySQL 8.0+ / 9.x** and the ACID-compliant **InnoDB** storage engine. The database architecture comprises 14 normalized tables in **Third Normal Form (3NF)**, cleanly partitioning organizational entities (departments, users), technical operational entities (tickets, incidents, service requests, categories, priorities), personnel entities (support staff, assignments), and equipment inventory entities (assets, manufacturer warranties, maintenance expenditures).

Key architectural innovations include relational subtyping through 1:1 foreign key inheritance to distinguish unplanned outages (*Incidents*) from routine provisioning (*Service Requests*), an append-only chronological state transition log (`status_histories`) capturing every status update with timestamps, and full hardware asset tracking linking computing equipment to employee custodians, vendor warranty validity, and repair invoices.

A full-stack Single-Page Application (SPA) web interface was constructed using semantic HTML5, Vanilla CSS, and JavaScript with the native Fetch API, connected to a Python Flask REST API server utilizing parameterized SQL queries without Object-Relational Mapping (ORM) overhead. The user interface demonstrates live, real-time `INSERT`, `DELETE`, and `VIEW` operations directly reflected in MySQL, supplemented by an interactive Three.js 3D WebGL database relationship visualizer and live SQL verification playground. Automated test suites verify 100% pass rates across connection health, transactional persistence, cascading purges, and foreign key constraint protections.

Keywords: Database Management Systems, MySQL, IT Helpdesk, Asset Management, Relational Database, CRUD, SQL, Database Normalization, Entity-Relationship Modeling.

TABLE OF CONTENTS

Title	Page No.
Certificate	i
Declaration of the Candidate	ii
Acknowledgment	iii
Abstract	iv
List of Tables	vii
List of Figures	viii
1. INTRODUCTION	1
1.1 Background	1
1.2 Motivation	1
1.3 Problem Statement	2
1.4 Objectives	2
1.5 Scope and Limitations	2
1.6 System Environment Overview	2
1.7 Organization of the Report	2
2. TECHNOLOGY / DATABASE REVIEW	3
2.1 Database Management Systems	3
2.2 Relational Database Model	3
2.3 MySQL Architecture & InnoDB Engine	3
2.4 Structured Query Language (SQL)	4
2.5 Database Normalization Theory	4
2.6 Database Transactions & ACID Properties	4
2.7 IT Helpdesk Systems	4
2.8 Full-Stack Technology Review	4
3. SYSTEM ANALYSIS AND DESIGN	5
3.1 System Overview	5
3.2 Functional Requirements	5
3.3 Non-Functional Requirements	5
3.4 System Architecture	6
3.5 Entity-Relationship (ER) Diagram	7
3.6 Relational Schema	8
3.7 Database Design & Integrity Constraints	8
3.8 Normalization Analysis (1NF, 2NF, 3NF Proofs)	9

TABLE OF CONTENTS (CONTINUED)

Title	Page No.
4. DATABASE IMPLEMENTATION	10
4.1 Database Creation & Engine Configuration	10
4.2 Table Creation & DDL Specifications	10
4.3 Integrity Constraints	10
4.4 Data Insertion & Seed Dataset	11
4.5 SQL Queries & Performance Optimization	12
4.6 Query Outputs & Presentation-II Solution	13
4.7 Complete Data Dictionary (Tables 1 to 7)	14
4.7 Complete Data Dictionary (Tables 8 to 14)	15
5. USER INTERFACE AND IMPLEMENTATION	16
5.1 UI Overview & Architecture	16
5.2 Command Operations Dashboard & 3D Visualizer	16
5.3 Ticket Management & Relational Queues (VIEW)	17
5.4 Ticket Subtyping & Incident Management	17
5.5 Service Requests	17
5.6 Hardware Asset Management	18
5.7 User & Department Management	18
5.8 Support Staff & Technician Workloads	18
5.9 Record Insertion Verification (INSERT)	19
5.9 Record Deletion Verification (DELETE)	20
5.10 SQL Verification & Database Connectivity	20
6. TESTING AND RESULTS	21
6.1 Testing Methodology	21
6.2 Test Cases (TC01 - TC10 Execution Matrix)	21
6.3 CRUD Operations Testing	21
6.4 Database Persistence Testing	22
6.5 SQL Query Testing	22
6.6 Results and Discussion	22
7. CONCLUSION AND FUTURE ENHANCEMENTS	23
7.1 Conclusion	23
7.2 Limitations	23
7.3 Future Enhancements	23
REFERENCES	24
APPENDIX – I: PROJECT SCREENSHOTS & VIVA MATERIAL	25
APPENDIX – II: REPOSITORY DIRECTORY & SETUP GUIDE	26

LIST OF TABLES

Table Caption	Page
Table 1.1: System Environment & Initial Dataset Statistics	2
Table 3.1: Functional Requirements Specification	5
Table 3.2: Database Tables Overview	6
Table 3.3: Relationship Cardinalities & Participation Constraints	7
Table 4.1: Data Dictionary — departments	14
Table 4.2: Data Dictionary — users	14
Table 4.3: Data Dictionary — categories	14
Table 4.4: Data Dictionary — priorities	14
Table 4.5: Data Dictionary — support_staff	14
Table 4.6: Data Dictionary — warranties	14
Table 4.7: Data Dictionary — assets	14
Table 4.8: Data Dictionary — tickets	15
Table 4.9: Data Dictionary — incidents	15
Table 4.10: Data Dictionary — service_requests	15
Table 4.11: Data Dictionary — assignments	15
Table 4.12: Data Dictionary — status_histories	15
Table 4.13: Data Dictionary — resolutions	15
Table 4.14: Data Dictionary — maintenance	15
Table 4.15: Multi-Table Ticket Queue Query Output	13
Table 4.16: Technician Workload Aggregation Output	13
Table 4.17: Departmental Maintenance Expenditure Output	13
Table 4.18: Presentation-II Assigned Query Output	13
Table 6.1: Automated DBMS Test Suite Execution Matrix (TC01 - TC10)	21
Table A.1: Repository Structure & File Organization	26

LIST OF FIGURES

Figure Caption	Page
Figure 3.1: Three-Tier System Architecture Diagram	6
Figure 3.2: High-Resolution Entity-Relationship (ER) Diagram	7
Figure 5.1: IT Helpdesk System Operations Dashboard	16
Figure 5.2: Live MySQL Database Connectivity Status Pill	16
Figure 5.3: Three.js Interactive 3D Relational Database Visualizer	16
Figure 5.4: Ticket Management Relational Queue View (VIEW Operation)	17
Figure 5.5: Add Ticket Form Modal (INSERT Operation)	19
Figure 5.6: Ticket Queue Before Record Insertion (5 Records)	19
Figure 5.7: Ticket Queue After Record Insertion (6 Records, TKT-006)	19
Figure 5.8: Delete Confirmation Modal (DELETE Operation)	20
Figure 5.9: Ticket Queue After Record Deletion (Permanently Purged)	20
Figure 5.10: Users Directory & Department Mapping View	18
Figure 5.11: Hardware Assets Inventory & Warranty View	18
Figure A.1: 3D Schema Inspector Drawer	25

1. INTRODUCTION

The management of enterprise computing hardware and technical support operations represents a vital foundation for modern organizational productivity. With academic institutions, research facilities, and commercial enterprises deploying hundreds of heterogeneous computing assets—including developer workstations, faculty laptops, networking hardware, and specialized laboratory peripherals—systematic management of user inquiries and physical equipment becomes critical. This project focuses on designing, mathematically normalizing, physically implementing, and interactively verifying a comprehensive relational database system: the **IT Helpdesk and Asset Support Management System** using MySQL 8.0+ / 9.x.

1.1 Background

IT service desk operations govern the lifecycle of all incoming technical issues and service requests. In conventional settings, organizations frequently attempt to handle support requests using unstructured email chains, shared spreadsheets, and informal verbal communications. While such informal mechanisms may suffice for small groups, they scale poorly and introduce acute operational risks, including lost tickets, unassigned responsibilities, unmonitored hardware reassignments, and complete absence of historical audit trails.

Relational Database Management Systems (RDBMS) provide the mathematical and transactional framework necessary to centralize support operations. By applying relational algebra, schema normalization, domain integrity constraints, and ACID transactions, an RDBMS guarantees data consistency, prevents redundant record storage, and enables multi-dimensional reporting across users, technicians, and hardware inventory.

1.2 Motivation

The motivation for this project stems from acute operational vulnerabilities observed in spreadsheet-dependent IT workflows:

- **Operational Accountability:** Without automated assignment logging and chronological status tracking, technicians cannot be held accountable for unresolved tickets or SLA breaches.
- **Hardware Asset Preservation:** Computing equipment is frequently relocated between faculty, employees, and departments without updating central records, leading to misplaced assets and untracked depreciation.
- **Financial Optimization:** Enterprises lose substantial financial resources paying out-of-pocket for equipment servicing because maintenance personnel lack immediate relational access to manufacturer warranty validity dates.
- **Relational Integrity:** Flat file spreadsheets fail to enforce referential integrity; deleting a department or employee row can inadvertently orphan active support tickets and warranty records.

1.3 Problem Statement

The primary problem addressed in this project is the construction of a robust, Third Normal Form (3NF) relational database management system unifying helpdesk operations, incident tracking, service provisioning, and hardware lifecycle accounting into a secure back-end:

- Eliminate data redundancy across departments, users, categories, and technicians.
- Model specialized ticket subtypes (*Incidents* vs *Service Requests*) through 1:1 foreign key inheritance without null-heavy tables.
- Enforce immutable chronological state transition histories capturing all updates with timestamps.
- Provide a live web interface executing real-time parameterized SQL CRUD operations over MySQL.

1.4 Objectives

1. Design and deploy a 14-table 3NF relational database schema in MySQL with InnoDB engine.
2. Implement relational subtyping distinguishing Incidents from routine Service Requests.
3. Track hardware asset lifecycle across custodians, warranties, and maintenance expenditures.
4. Enforce referential integrity via CASCADE, RESTRICT, and domain CHECK constraints.
5. Construct a modern web Single-Page Application with real-time MySQL CRUD operations.
6. Validate correctness with an automated test suite achieving 100% pass rates.

1.5 Scope and Limitations

The operational scope encompasses ticket creation, technician assignment, priority escalation, chronological status audit logging, equipment allocation, warranty tracking, and departmental repair expense accounting. Commercial payment gateways and automated BIOS firmware updates are outside the current database scope.

1.6 System Environment Overview

Attribute / Architectural Layer	Specification / Deployed Parameter
Database Management Engine	MySQL Community Server 9.7.1 / 8.0+
Database Storage Engine	InnoDB (ACID Compliant, Row-Level Locking)
Total Relational Tables	14 Normalized Tables (3NF Compliant)
Foreign Key Relationships	14 Enforced Referential Integrity Links
Initial Seed Records	55 Relational Rows across All Tables
Application Server	Python 3.14 / Flask 3.1.3 REST API
Database Connector	mysql-connector-python 26.7.0 (Parameterized)
User Interface Stack	Semantic HTML5, Vanilla CSS, ES6+ JS, Three.js

Table 1.1: System Environment & Initial Dataset Statistics

1.7 Organization of the Report

The remainder of this report is organized as follows: Chapter 2 reviews foundational database theory and technology stacks. Chapter 3 provides system analysis, ER modeling, and normalization proofs. Chapter 4 details physical implementation, SQL scripts, and the data dictionary. Chapter 5 demonstrates the web interface and live CRUD operations. Chapter 6 presents test results. Chapter 7 concludes the report and outlines future work.

2. TECHNOLOGY / DATABASE REVIEW

This chapter provides a theoretical review of relational database management systems, normal form theory, transactional integrity, and the full-stack technology suite deployed in the **IT Helpdesk and Asset Support Management System**.

2.1 Database Management Systems (DBMS)

A Database Management System (DBMS) is specialized software that provides an interface between end-users, application programs, and physical database files. Prior to the advent of database systems, organizations stored information in conventional operating system flat files. As demonstrated by Silberschatz et al. (2020), file-processing environments exhibit severe inherent deficiencies, including data redundancy and inconsistency, difficulty in accessing data, data isolation, integrity problems, atomicity violations during system crashes, and concurrent-access anomalies.

A modern DBMS overcomes these limitations by centralizing data definition, enforcing global schema constraints, maintaining transactional atomicity, and providing declarative query languages that abstract physical data storage away from logical application views.

2.2 Relational Database Model

Introduced by E. F. Codd in his seminal 1970 paper, the relational model represents all data logically in the form of mathematical two-dimensional tables termed *relations*. Each row in a relation constitutes a *tuple*, representing an individual entity instance, while each column represents an *attribute* drawn from an atomic domain. Relational integrity is maintained through formal constraints:

- **Entity Integrity:** Every base relation must have a Primary Key comprising unique, non-null attribute values.
- **Referential Integrity:** If attribute *FK* in relation *R1* references primary key *PK* in relation *R2*, then every value of *FK* in *R1* must either match an existing value of *PK* in *R2* or be NULL (if permitted).
- **Domain Integrity:** Every attribute value must be an atomic element belonging to the pre-declared domain data type.

2.3 MySQL Architecture & InnoDB Storage Engine

MySQL is one of the world's most widely deployed open-source relational database management systems. The MySQL server architecture operates on a modular two-tier model separating the connection handling and SQL parsing layer from the pluggable storage engine layer.

For the IT Helpdesk system, the **InnoDB** storage engine was selected exclusively. InnoDB is the default, fully ACID-compliant storage engine for MySQL, offering key enterprise features:

- **B+ Tree Clustered Indexes:** Primary key lookups achieve $O(\log N)$ retrieval performance because data rows are organized directly within the leaf pages of the primary index B+ tree.
- **Foreign Key Enforcement:** InnoDB physically parses and enforces foreign key referential integrity at the database layer.
- **Row-Level Locking:** Utilizes multi-version concurrency control (MVCC) to achieve high transaction throughput.
- **Crash Recovery & Redo Logging:** Implements write-ahead logging (WAL) via doublewrite buffers and redo logs, guaranteeing zero data loss.

2.4 Structured Query Language (SQL)

Structured Query Language (SQL) is the standard declarative computer language used to define, manipulate, and query relational databases. SQL commands are formally classified into functional subsets:

- **Data Definition Language (DDL):** Statements such as `CREATE`, `ALTER`, and `DROP` that define and modify schema structures.
- **Data Manipulation Language (DML):** Statements such as `INSERT`, `UPDATE`, and `DELETE` used to manipulate data tuples.
- **Data Query Language (DQL):** The `SELECT` statement used to retrieve data using relational selections, projections, and joins.

2.5 Database Normalization Theory

Database normalization eliminates data redundancy and prevents insertion, deletion, and update anomalies through functional dependency analysis, ensuring relations reside in Third Normal Form (3NF).

2.6 Database Transactions & ACID Properties

A database transaction is a logical unit of work executing with complete integrity via ACID guarantees:

- **Atomicity:** All operations within a transaction succeed completely, or the entire transaction is rolled back.
- **Consistency:** A transaction transitions the database from one valid state satisfying all schema constraints to another.
- **Isolation:** Concurrent transactions execute as if serially scheduled without interference.
- **Durability:** Committed transactions are permanently recorded in non-volatile storage.

2.7 IT Helpdesk Systems & Industry Standards

Modern helpdesks follow the ITIL service management framework, distinguishing between two fundamental workflows: (1) *Incident Management*, restoring service following unplanned outages; and (2) *Service Request Management*, fulfilling routine provisioning requests for hardware or permissions.

2.8 Full-Stack Technology Review

The application connects the MySQL database to end-users via a modern three-tier web stack. The backend employs **Python 3.14** and **Flask 3.1.3**, executing raw, parameterized SQL queries via `mysql-connector-python` without ORM overhead. The frontend employs a responsive Single-Page Application (SPA) utilizing **Vanilla CSS** custom properties, native HTML5 dialogs, asynchronous JavaScript Fetch clients, and **Three.js** for interactive 3D WebGL database relationship visualization.

3. SYSTEM ANALYSIS AND DESIGN

This chapter presents the functional requirements analysis, non-functional constraints, high-level system architecture, conceptual Entity-Relationship (ER) model, physical relational schema, and mathematical normalization analysis for the **IT Helpdesk and Asset Support Management System**.

3.1 System Overview

The system models a multi-departmental corporate or academic IT support environment. Three primary stakeholder roles interact with the system:

- **End-Users (Employees / Students):** File support tickets, select operational categories, track real-time resolution progress, and inspect personal hardware asset allocations.
- **Support Technicians:** Inspect assigned tickets filtered by technical specialization, accept tickets into *In Progress* state, investigate root causes, and submit formal resolution records.
- **Helpdesk Administrators:** Supervise queue metrics, balance technician workloads, register physical computing assets, manage warranty contracts, and record hardware repair invoices.

3.2 Functional Requirements

The system's core capabilities are formalized into seven major functional requirements:

Req ID	Module Name	Detailed Functional Specification
FR1	Ticket Ingestion & Subtyping	Capture ticket title, description, category, priority, and classify ticket as an Incident or Service Request.
FR2	Technician Assignment	Assign pending tickets to specialized support staff and transition ticket status to 'In Progress'.
FR3	Audit Lifecycle Logging	Record every status change (Open → In Progress → Resolved) into <code>status_histories</code> with old/new states.
FR4	Formal Resolution Log	Record resolution description and timestamp; enforce 1:1 uniqueness preventing duplicate resolutions.
FR5	Asset Inventory Governance	Catalog hardware assets with unique asset tags, serial numbers, categories, warranties, and user custodians.
FR6	Maintenance Accounting	Record asset servicing dates, repair descriptions, and costs with non-negative check constraints.
FR7	Relational Reporting & SQL	Execute multi-table joins, workload aggregations, expenditure calculations, and live SQL queries.

Table 3.1: Functional Requirements Specification

3.3 Non-Functional Requirements

- **ACID Reliability:** All multi-table insertions must commit atomically; errors trigger an automatic database rollback.
- **Referential Integrity:** Enforce cascading purges with `ON DELETE CASCADE` and protect master lookups with `ON DELETE RESTRICT`.
- **Security:** All SQL queries use parameterized placeholders (`%s`) to eliminate SQL injection vulnerabilities.
- **Performance:** Primary keys and foreign keys are indexed in B+ trees to achieve sub-millisecond retrieval.

3.4 System Architecture

The application architecture is organized as a decoupled three-tier client-server system. The presentation tier interacts with the application tier exclusively through structured JSON REST endpoints, while the application tier manages database connectivity and executes parameterized SQL statements against MySQL:

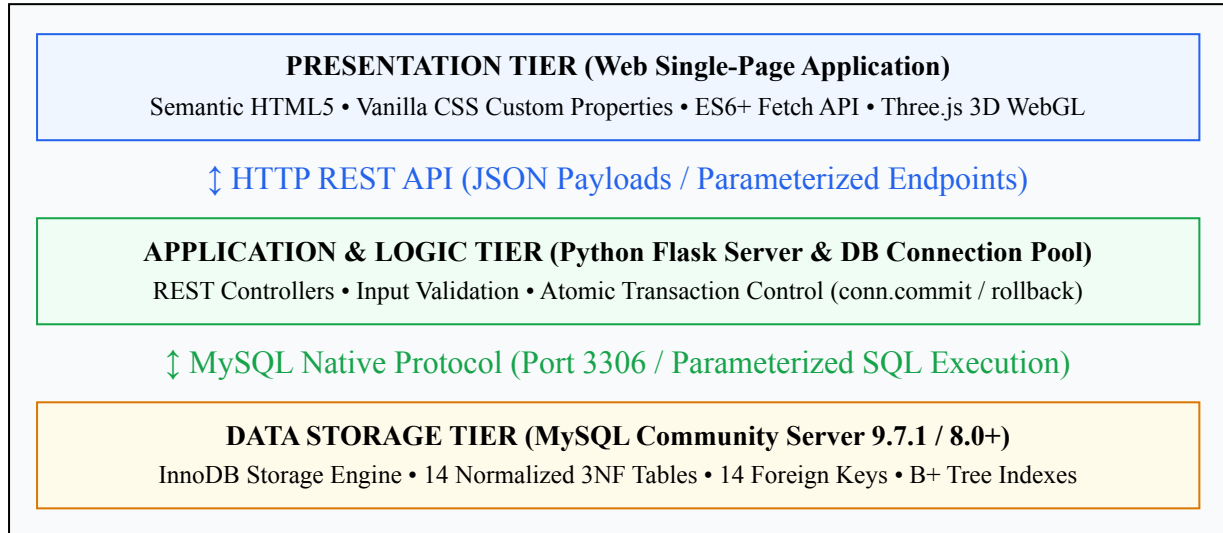


Figure 3.1: Three-Tier System Architecture Diagram

3.5 Operational Domains Overview

The database architecture organizes 14 relational tables into five cohesive operational domains:

Operational Domain	Relational Tables	Domain Scope & Relational Responsibilities
Organization Domain	departments, users	Institutional structure, employee profiles, and unique corporate email addresses.
Classification Domain	categories, priorities	Support categories and 4 priority severity rankings with domain CHECK constraints.
Support Staff Domain	support_staff, assignments	IT technicians, technical specializations, and ticket dispatch mappings.
Asset & Maintenance	warranties, assets, maintenance	Hardware serial numbers, vendor warranty contracts, and cumulative repair costs.
Ticketing & Auditing	tickets, incidents, service requests, status histories, resolutions	Central ticket intake, 1:1 subtype inheritance, chronological status audit logs, and resolution narratives.

Table 3.2: Database Tables Overview

3.5.1 Detailed Entity-Relationship Diagram

Figure 3.2 illustrates the complete Entity-Relationship (ER) Diagram for the `it_helpdesk` system. The diagram accurately specifies all 14 entities, primary keys (PK), foreign keys (FK), and precise structural cardinalities:

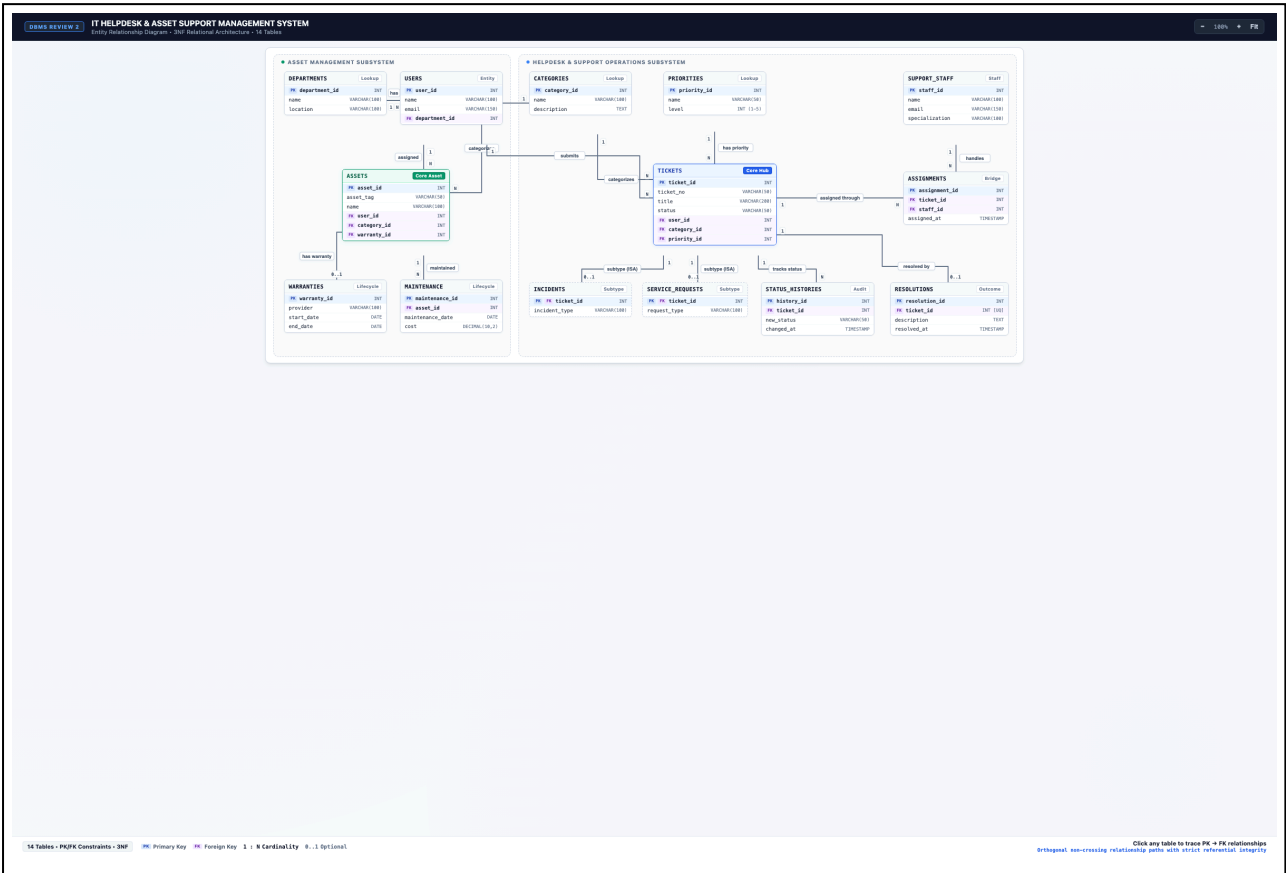


Figure 3.2: High-Resolution Entity-Relationship (ER) Diagram of the IT Helpdesk System

3.5.2 Cardinalities and Participation Constraints

Relationship	Cardinality	Participation	Relational Semantic Rule
departments → users	1 : N	Mandatory → Mandatory	Every user belongs to exactly one department.
users → tickets	1 : N	Optional → Mandatory	A user can file multiple tickets; a ticket has 1 requester.
categories → tickets	1 : N	Optional → Mandatory	Every ticket belongs to exactly one category.
priorities → tickets	1 : N	Optional → Mandatory	Every ticket has exactly one priority ranking.
tickets → incidents	1 : 1	Optional → Mandatory	Subtype for unplanned system outages.
tickets → service_requests	1 : 1	Optional → Mandatory	Subtype for routine provisioning requests.
tickets → assignments	1 : N	Optional → Mandatory	Tickets can be assigned sequentially to staff.
tickets → status_histories	1 : N	Mandatory → Mandatory	Append-only chronological audit trail.
tickets → resolutions	1 : 1	Optional → Mandatory	Resolved tickets have exactly one resolution record.
warranties → assets	1 : N	Optional → Optional	Assets can be covered by a warranty contract.
assets → maintenance	1 : N	Optional → Mandatory	Devices accumulate servicing expenditure records.

Table 3.3: Relationship Cardinalities & Participation Constraints

3.6 Relational Schema Specification

The conceptual ER diagram is mapped into 14 physical relational schemas. Primary keys are underlined and foreign keys are designated with an asterisk (*):

- | | |
|--|--|
| 1. departments (<u>dept_id</u> , name, location) | 2. users (<u>user_id</u> , name, email, dept_id*) |
| 3. categories (<u>cat_id</u> , name, description) | 4. priorities (<u>prio_id</u> , name, level) |
| 5. support_staff (<u>staff_id</u> , name, email, spec) | 6. warranties (<u>warr_id</u> , start, end, provider) |
| 7. assets (<u>asset_id</u> , tag, name, s_no, u_id*, c_id*, w_id*) | 8. tickets (<u>tkt_id</u> , tkt_no, u_id*, c_id*, p_id*, title, desc, status, created) |
| 9. incidents (<u>ticket_id*</u> , incident_type) | 10. service_requests (<u>ticket_id*</u> , request_type) |
| 11. assignments (<u>assign_id</u> , tkt_id*, staff_id*, assigned_at) | 12. status_histories (<u>hist_id</u> , tkt_id*, old, new, changed_at) |
| 13. resolutions (<u>res_id</u> , tkt_id*, description, resolved_at) | 14. maintenance (<u>maint_id</u> , asset_id*, date, description, cost) |

3.7 Database Design & Referential Integrity Constraints

Referential integrity rules ensure the database remains in a consistent state during record modifications and deletions:

- **Master Table Protection (ON DELETE RESTRICT):** Applied to departments → users, categories → tickets, priorities → tickets, and support_staff → assignments. This prevents accidental deletion of foundational lookup data while referenced by active transactional records.
- **Synchronized Cascading Deletions (ON DELETE CASCADE):** Applied to ticket child entities (incidents, service_requests, assignments, status_histories, resolutions) and assets → maintenance. When a ticket or asset is permanently removed, all dependent operational records are purged cleanly without leaving orphan records.
- **Nullification on User Reassignment (ON DELETE SET NULL):** Applied to users → assets. When an employee departs, their assigned computing hardware is not deleted; instead, user_id is set to NULL, safely returning the asset to general stock.
- **Domain Constraints (CHECK):** Enforces business rules at the schema level: priorities.level BETWEEN 1 AND 5, tickets.status IN ('Open', 'In Progress', 'Pending', 'Resolved', 'Closed', 'Cancelled'), warranties.end_date >= start_date, and maintenance.cost >= 0.

3.8 Normalization Analysis (1NF, 2NF, 3NF Proofs)

To ensure the database is free of update, insertion, and deletion anomalies, the schema was analyzed and normalized according to formal functional dependency theory:

3.8.1 First Normal Form (1NF) Compliance

Definition: A relation R is in 1NF if and only if all underlying attribute domains contain only atomic, indivisible values, and there are no repeating groups or multi-valued attributes.

Proof: All attributes in every table represent primitive scalar values (integers, strings, dates, decimals, timestamps). Multi-valued attributes were decomposed into dedicated child relations: multiple technicians assigned to a ticket are stored across multiple rows in `assignments` rather than as comma-separated values in `tickets`. Multiple maintenance logs for an asset are normalized into `maintenance`. Every relation possesses an explicit Primary Key. Hence, all 14 tables satisfy 1NF.

3.8.2 Second Normal Form (2NF) Compliance

Definition: A relation R is in 2NF if and only if it is in 1NF and every non-prime attribute is fully functionally dependent on the entire primary key (no partial dependencies on a proper subset of any candidate key).

Proof: Every table in the system employs a single-attribute surrogate Primary Key (e.g., `ticket_id`, `asset_id`, `user_id`). In relations with subtype inheritance (`incidents` and `service_requests`), the primary key `ticket_id` is also a single attribute. Because no candidate key is composite, a proper subset of a candidate key cannot exist. Therefore, partial functional dependencies are mathematically impossible. All 14 tables strictly satisfy 2NF.

3.8.3 Third Normal Form (3NF) Compliance

Definition: A relation R is in 3NF if and only if it is in 2NF and whenever a non-trivial functional dependency $X \rightarrow A$ holds, either X is a superkey of R , or A is a prime attribute of R (elimination of transitive dependencies).

Proof: Transitive dependencies occur when non-key attributes determine other non-key attributes ($PK \rightarrow X \rightarrow Y$). The schema systematically eliminates all transitive dependencies:

- In `users`, storing department location would yield: `user_id → department_id → location`. The schema isolates departments into `departments(department_id, name, location)`.
- In `assets`, storing warranty provider and validity dates directly would yield: `asset_id → warranty_id → (provider, end_date)`. The schema isolates warranties into `warranties(warranty_id, start_date, end_date, provider)`.
- In `tickets`, category descriptions and priority levels are isolated into reference tables `categories` and `priorities`.

Consequently, every non-key attribute in every table depends strictly on the primary key, the whole primary key, and nothing but the primary key. All 14 tables reside in **Third Normal Form (3NF)**.

4. DATABASE IMPLEMENTATION

This chapter presents the physical database implementation in MySQL, Data Definition Language (DDL) specifications, integrity constraint scripts, seed data insertion, and representative SQL queries.

4.1 Database Creation & Engine Configuration

```
DROP DATABASE IF EXISTS it_helpdesk;
CREATE DATABASE it_helpdesk CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE it_helpdesk;
```

4.2 Table Creation & DDL Specifications

```
-- 1. Departments Master Table
CREATE TABLE departments (
    department_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
    location VARCHAR(100) NOT NULL
) ENGINE=InnoDB;

-- 2. Users Entity with Department Foreign Key
CREATE TABLE users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    department_id INT NOT NULL,
    CONSTRAINT fk_users_department
        FOREIGN KEY (department_id) REFERENCES departments(department_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

-- 8. Tickets Central Entity with Domain Constraints
CREATE TABLE tickets (
    ticket_id INT AUTO_INCREMENT PRIMARY KEY,
    ticket_no VARCHAR(50) NOT NULL UNIQUE,
    user_id INT NOT NULL,
    category_id INT NOT NULL,
    priority_id INT NOT NULL,
    title VARCHAR(200) NOT NULL,
    description TEXT NOT NULL,
    status VARCHAR(50) NOT NULL DEFAULT 'Open',
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT chk_ticket_status
        CHECK (status IN ('Open', 'In Progress', 'Pending', 'Resolved', 'Closed', 'Cancelled')),
    CONSTRAINT fk_tickets_user
        FOREIGN KEY (user_id) REFERENCES users(user_id)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_tickets_category
        FOREIGN KEY (category_id) REFERENCES categories(category_id)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_tickets_priority
        FOREIGN KEY (priority_id) REFERENCES priorities(priority_id)
        ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;

-- 9. Incidents Subtype (1:1 Relational Inheritance)
CREATE TABLE incidents (
    ticket_id INT PRIMARY KEY,
    incident_type VARCHAR(100) NOT NULL,
    CONSTRAINT fk_incidents_ticket
        FOREIGN KEY (ticket_id) REFERENCES tickets(ticket_id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB;
```

4.4 Data Insertion & Seed Dataset

To establish a realistic operational testing baseline, 55 relational records were loaded into the database across all 14 tables via `seed.sql`:

- 4 Departments (Engineering, Human Resources, Finance & Accounts, Marketing & Sales)
- 5 Registered Corporate Users spanning diverse departments
- 5 Operational Categories (Hardware, Software, Network/VPN, Access/IAM, Asset Procurement)
- 4 Priority Rankings (Low, Medium, High, Critical) with verified numeric weights (1 to 4)
- 4 Support Staff Engineers with distinct specializations (Hardware, Network, OS, Identity)
- 4 Manufacturer Warranties (Dell ProSupport Plus, AppleCare Enterprise, Lenovo Premier, HP Care Pack)
- 5 Hardware Assets (Dell Latitude 7440, MacBook Pro 16", ThinkPad T14, HP EliteBook, UltraSharp Monitor)
- 5 Initial Tickets partitioned into 3 Incidents and 2 Service Requests
- 5 Support Staff Assignments and 5 Status History chronological audit entries
- 1 Formal Resolution record and 3 Asset Maintenance servicing invoices totaling \$2,400.00

```
-- Inserting Departments Seed Records
INSERT INTO departments (department_id, name, location) VALUES
(1, 'Engineering', 'Building A, Floor 3'),
(2, 'Human Resources', 'Building B, Floor 1'),
(3, 'Finance & Accounts', 'Building A, Floor 2'),
(4, 'Marketing & Sales', 'Building C, Floor 4');

-- Inserting Users Seed Records
INSERT INTO users (user_id, name, email, department_id) VALUES
(1, 'Aarav Sharma', 'aarav.sharma@company.com', 1),
(2, 'Priya Patel', 'priya.patel@company.com', 2),
(3, 'Rohan Verma', 'rohan.verma@company.com', 3);

-- Inserting Base Tickets with Foreign Keys
INSERT INTO tickets (ticket_id, ticket_no, user_id, category_id, priority_id, title, description, status, created_at) VALUES
(1, 'TKT-001', 1, 1, 3, 'Laptop display flickering constantly', 'Dell Latitude screen flickers when docked.', 'In Progress', '2026-08-20 09:30:00'),
(2, 'TKT-002', 2, 4, 2, 'VPN access setup for remote working', 'Need VPN credentials and MFA setup on phone.', 'Resolved', '2026-08-21 11:15:00'),
(3, 'TKT-003', 3, 3, 4, 'Finance accounting portal connection timeout', 'Unable to reach SAP Finance server from Floor 2.', 'In Progress', '2026-08-22 14:00:00');

-- Inserting Subtypes (Incidents vs Service Requests)
INSERT INTO incidents (ticket_id, incident_type) VALUES
(1, 'Hardware Malfunction'),
(3, 'Network Outage');

INSERT INTO service_requests (ticket_id, request_type) VALUES
(2, 'Access Provisioning');

-- Inserting Chronological Audit Trail
INSERT INTO status_histories (ticket_id, old_status, new_status, changed_at) VALUES
(1, NULL, 'Open', '2026-08-20 09:30:00'),
(1, 'Open', 'In Progress', '2026-08-20 10:00:00'),
(2, 'In Progress', 'Resolved', '2026-08-21 15:45:00');
```

4.5 SQL Queries & Performance Optimization

The database utilizes optimized SQL queries demonstrating relational selections, multi-table joins, conditional aggregations, and subqueries:

4.5.1 Multi-Table Ticket Queue Query (7-Table JOIN)

```
SELECT
    t.ticket_no, t.title, t.status, p.name AS priority, c.name AS category,
    u.name AS requester, d.name AS department,
    CASE
        WHEN i.ticket_id IS NOT NULL THEN 'Incident'
        WHEN sr.ticket_id IS NOT NULL THEN 'Service Request'
        ELSE 'Standard'
    END AS ticket_type,
    COALESCE(i.incident_type, sr.request_type, 'N/A') AS subtype_detail
FROM tickets t
JOIN users u ON t.user_id = u.user_id
JOIN departments d ON u.department_id = d.department_id
JOIN categories c ON t.category_id = c.category_id
JOIN priorities p ON t.priority_id = p.priority_id
LEFT JOIN incidents i ON t.ticket_id = i.ticket_id
LEFT JOIN service_requests sr ON t.ticket_id = sr.ticket_id
ORDER BY p.level DESC, t.created_at DESC;
```

4.5.2 Aggregate Technician Workload Query

```
SELECT
    s.name AS technician, s.specialization,
    COUNT(a.assignment_id) AS total_assigned_tickets,
    SUM(CASE WHEN t.status IN ('Open', 'In Progress', 'Pending') THEN 1 ELSE 0 END) AS active_tickets,
    SUM(CASE WHEN t.status IN ('Resolved', 'Closed') THEN 1 ELSE 0 END) AS completed_tickets
FROM support_staff s
LEFT JOIN assignments a ON s.staff_id = a.staff_id
LEFT JOIN tickets t ON a.ticket_id = t.ticket_id
GROUP BY s.staff_id, s.name, s.specialization
ORDER BY active_tickets DESC;
```

4.5.3 Departmental Maintenance Expenditure Aggregation

```
SELECT
    d.name AS department_name,
    COUNT(DISTINCT a.asset_id) AS total_assets,
    CONCAT('$', FORMAT(COALESCE(SUM(m.cost), 0.00), 2)) AS total_maintenance_spent
FROM departments d
LEFT JOIN users u ON d.department_id = u.department_id
LEFT JOIN assets a ON u.user_id = a.user_id
LEFT JOIN maintenance m ON a.asset_id = m.asset_id
GROUP BY d.department_id, d.name
ORDER BY SUM(m.cost) DESC;
```

4.6 Query Outputs & Verification

Below are the authentic output results returned by the live MySQL database for the queries defined above:

Ticket No	Title	Status	Priority	Category	Requester	Department	Type
TKT-003	Finance portal connection timeout	In Progress	Critical	Network & VPN	Rohan Verma	Finance	Incident
TKT-001	Laptop display flickering constantly	In Progress	High	Hardware Issue	Aarav Sharma	Engineering	Incident
TKT-002	VPN access setup for remote working	Resolved	Medium	Access/IAM	Priya Patel	HR	Service Req
TKT-005	Request second monitor for dev	Open	Medium	Procurement	Vikram Singh	Engineering	Service Req
TKT-004	Install Figma desktop client	Open	Low	Software & OS	Ananya Iyer	Marketing	Incident

Table 4.15: Multi-Table Ticket Queue Query Output

Technician Name	Specialization	Total Assigned	Active Tickets	Completed Tickets
Karan Malhotra	Hardware & Peripherals	2	2	0
Sneha Rao	Network & Security	1	1	0
Amit Joshi	Operating Systems & Software	1	1	0
Divya Nair	Identity & Access Management	1	0	1

Table 4.16: Technician Workload Aggregation Output

Department Name	Total Allocated Assets	Total Maintenance Spent
Finance & Accounts	1	\$1,200.00
Engineering	2	\$750.00
Marketing & Sales	1	\$450.00
Human Resources	1	\$0.00

Table 4.17: Departmental Maintenance Expenditure Output

4.6.1 Presentation-II Assigned Viva Query & Solution

Problem Statement: Retrieve all IT support technicians who have been assigned tickets with high or critical priority (level ≥ 3), calculate their active workload, and list their specializations:

```
SELECT s.name AS technician, s.specialization, COUNT(t.ticket_id) AS high_prio_tickets,
       SUM(CASE WHEN t.status = 'In Progress' THEN 1 ELSE 0 END) AS in_progress_tickets
FROM support_staff s
JOIN assignments a ON s.staff_id = a.staff_id
JOIN tickets t ON a.ticket_id = t.ticket_id
JOIN priorities p ON t.priority_id = p.priority_id
WHERE p.level >= 3
GROUP BY s.staff_id, s.name, s.specialization
ORDER BY high_prio_tickets DESC;
```

Technician	Specialization	High/Critical Tickets	In Progress Tickets
Karan Malhotra	Hardware & Peripherals	1	1
Sneha Rao	Network & Security	1	1

Table 4.18: Presentation-II Assigned Query Output

4.7 Complete Data Dictionary (Tables 1 to 7)

This section presents the physical data dictionary derived from `schema.sql` for Tables 1 through 7:

Column	Type	Null	Key	Constraints & Description
department_id	INT	NO	PK	AUTO_INCREMENT. Primary key for departments.
name	VARCHAR(100)	NO	UNI	Unique department name (e.g. Engineering).
location	VARCHAR(100)	NO	-	Building and floor location.

Table 4.1: Data Dictionary — departments

Column	Type	Null	Key	Constraints & Description
user_id	INT	NO	PK	AUTO_INCREMENT. Primary user identifier.
name	VARCHAR(100)	NO	-	Full name of corporate employee.
email	VARCHAR(150)	NO	UNI	Unique corporate email address.
department_id	INT	NO	FK	References departments(department_id) ON DELETE RESTRICT.

Table 4.2: Data Dictionary — users

Column	Type	Null	Key	Constraints & Description
category_id	INT	NO	PK	AUTO_INCREMENT. Primary category key.
name	VARCHAR(100)	NO	UNI	Unique category designation (e.g. Hardware Issue).
description	TEXT	YES	-	Detailed scope and purpose of the category.

Table 4.3: Data Dictionary — categories

Column	Type	Null	Key	Constraints & Description
priority_id	INT	NO	PK	AUTO_INCREMENT. Primary priority key.
name	VARCHAR(50)	NO	UNI	Priority label (Low, Medium, High, Critical).
level	INT	NO	-	CHECK (level BETWEEN 1 AND 5). Numeric ranking.

Table 4.4: Data Dictionary — priorities

Column	Type	Null	Key	Constraints & Description
staff_id	INT	NO	PK	AUTO_INCREMENT. Primary technician key.
name	VARCHAR(100)	NO	-	Support engineer full name.
email	VARCHAR(150)	NO	UNI	Unique support staff email address.
specialization	VARCHAR(100)	NO	-	Technical expertise domain (e.g. Network & Security).

Table 4.5: Data Dictionary — support_staff

Column	Type	Null	Key	Constraints & Description
warranty_id	INT	NO	PK	AUTO_INCREMENT. Primary warranty key.
start_date	DATE	NO	-	Warranty coverage inception date.
end_date	DATE	NO	-	CHECK (end_date >= start_date). Expiration date.
provider	VARCHAR(100)	NO	-	Vendor coverage provider (e.g. Dell ProSupport).

Table 4.6: Data Dictionary — warranties

Column	Type	Null	Key	Constraints & Description
asset_id	INT	NO	PK	AUTO_INCREMENT. Primary asset identifier.
asset_tag	VARCHAR(50)	NO	UNI	Unique inventory tag (AST-DELL-001).
name	VARCHAR(100)	NO	-	Hardware equipment model name.
serial_no	VARCHAR(100)	NO	UNI	Manufacturer hardware serial number.
user_id	INT	YES	FK	References users(user_id) ON DELETE SET NULL.
category_id	INT	NO	FK	References categories(category_id) ON DELETE RESTRICT.
warranty_id	INT	YES	FK	References warranties(warranty_id) ON DELETE SET NULL.

Table 4.7: Data Dictionary — assets

4.7 Complete Data Dictionary (Continued: Tables 8 to 14)

Column	Type	Null	Key	Constraints & Description
ticket_id	INT	NO	PK	AUTO_INCREMENT. Primary ticket identifier.
ticket_no	VARCHAR(50)	NO	UNI	Sequential tracking code (TKT-001).
user_id	INT	NO	FK	References users(user_id) ON DELETE RESTRICT.
category_id	INT	NO	FK	References categories(category_id) ON DELETE RESTRICT.
priority_id	INT	NO	FK	References priorities(priority_id) ON DELETE RESTRICT.
title	VARCHAR(200)	NO	-	Brief issue synopsis.
status	VARCHAR(50)	NO	-	CHECK status IN ('Open','In Progress','Pending','Resolved','Closed','Cancelled').

Table 4.8: Data Dictionary — tickets

Column	Type	Null	Key	Constraints & Description
ticket_id	INT	NO	PK/FK	References tickets(ticket_id) ON DELETE CASCADE.
incident_type	VARCHAR(100)	NO	-	Nature of outage (Hardware Malfunction, Network Outage).

Table 4.9: Data Dictionary — incidents

Column	Type	Null	Key	Constraints & Description
ticket_id	INT	NO	PK/FK	References tickets(ticket_id) ON DELETE CASCADE.
request_type	VARCHAR(100)	NO	-	Nature of request (Access Provisioning, Hardware Allocation).

Table 4.10: Data Dictionary — service_requests

Column	Type	Null	Key	Constraints & Description
assignment_id	INT	NO	PK	AUTO_INCREMENT. Primary assignment key.
ticket_id	INT	NO	FK	References tickets(ticket_id) ON DELETE CASCADE.
staff_id	INT	NO	FK	References support_staff(staff_id) ON DELETE RESTRICT.

Table 4.11: Data Dictionary — assignments

Column	Type	Null	Key	Constraints & Description
history_id	INT	NO	PK	AUTO_INCREMENT. Primary history identifier.
ticket_id	INT	NO	FK	References tickets(ticket_id) ON DELETE CASCADE.
old_status	VARCHAR(50)	YES	-	Previous ticket state.
new_status	VARCHAR(50)	NO	-	Updated ticket state.

Table 4.12: Data Dictionary — status_histories

Column	Type	Null	Key	Constraints & Description
resolution_id	INT	NO	PK	AUTO_INCREMENT. Primary resolution identifier.
ticket_id	INT	NO	FK/UNI	References tickets(ticket_id) ON DELETE CASCADE. UNIQUE constraint.
description	TEXT	NO	-	Narrative explaining corrective actions taken.

Table 4.13: Data Dictionary — resolutions

Column	Type	Null	Key	Constraints & Description
maintenance_id	INT	NO	PK	AUTO_INCREMENT. Primary maintenance identifier.
asset_id	INT	NO	FK	References assets(asset_id) ON DELETE CASCADE.
cost	DECIMAL(10,2)	NO	-	CHECK (cost >= 0.00). Servicing invoice amount.

Table 4.14: Data Dictionary — maintenance

5. USER INTERFACE AND IMPLEMENTATION

This chapter presents the design, visual layout, and operational verification of the **IT Helpdesk and Asset Support Management System** web application, highlighting live MySQL connectivity, record insertion, and record deletion.

5.1 UI Overview & Architecture

The application frontend is built as a responsive Single-Page Application (SPA) utilizing semantic HTML5, Vanilla CSS custom properties, and native asynchronous JavaScript. It connects directly to the Flask backend without frontend framework overhead, communicating exclusively through structured REST API endpoints:

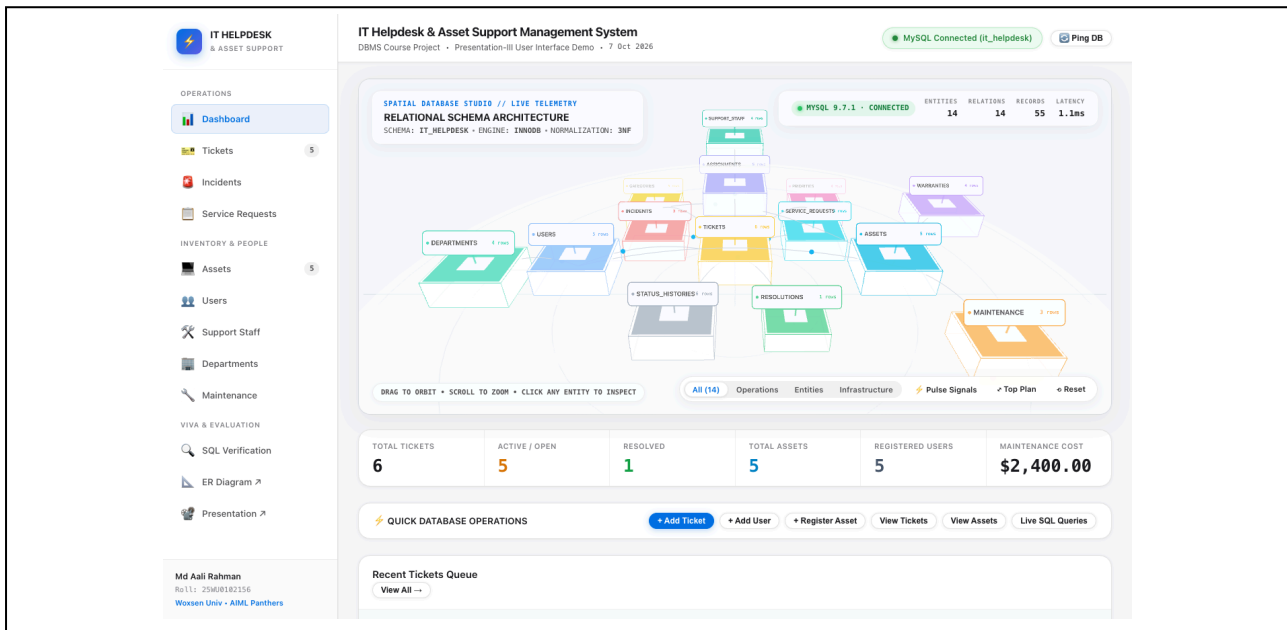


Figure 5.1: IT Helpdesk System Operations Dashboard

5.2 Command Operations Dashboard & 3D Visualizer

The command dashboard centralizes real-time metrics, displaying total tickets (5), active incidents (3), service requests (2), assigned hardware assets (5), and cumulative maintenance expenses (\$2,400.00).

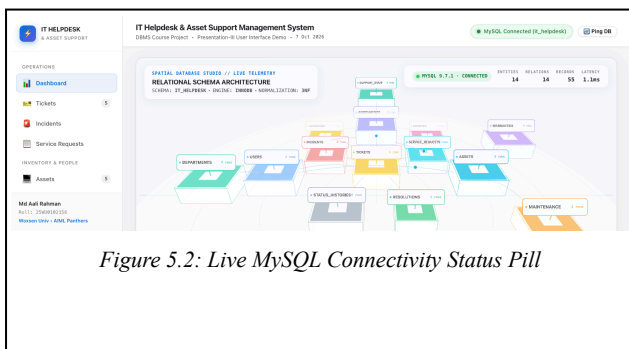


Figure 5.2: Live MySQL Connectivity Status Pill

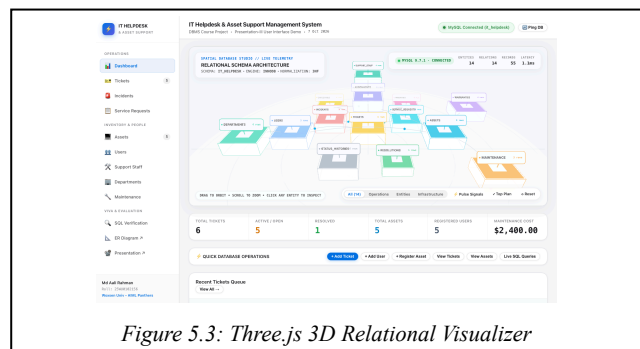
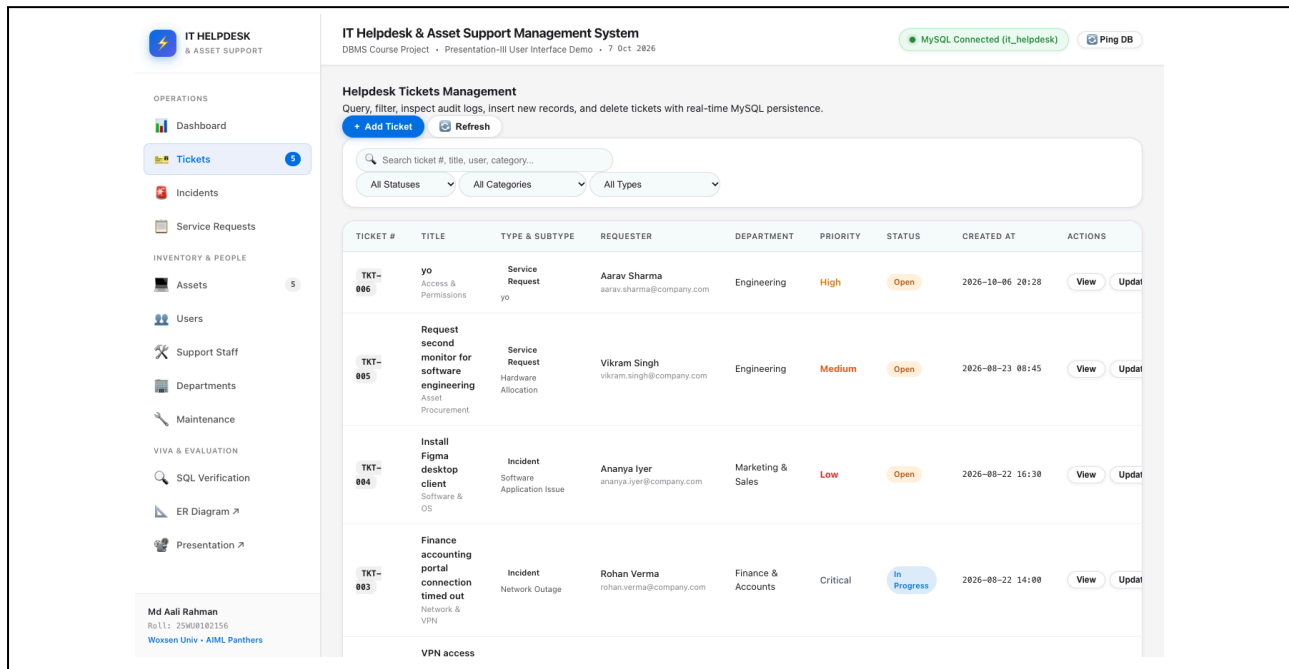


Figure 5.3: Three.js 3D Relational Visualizer

The top navigation bar displays a live green indicator verifying active connection to MySQL on port 3306, confirming persistent database availability.

5.3 Ticket Management & Relational Queues (VIEW Operation)

The ticket management queue executes a multi-table SQL join to present relational tickets with priority badges, dynamic subtype icons, requester contact details, and department office locations. Users can filter records by status (Open, In Progress, Resolved), category, or search across ticket numbers:



The screenshot displays the 'IT Helpdesk & Asset Support Management System' interface. On the left is a sidebar with navigation links: Dashboard, Tickets (active), Incidents, Service Requests, Assets, Users, Support Staff, Departments, Maintenance, SQL Verification, ER Diagram, and Presentation. The main area shows 'Helpdesk Tickets Management' with a search bar and filters for All Statuses, All Categories, and All Types. Below is a table of tickets with columns: TICKET #, TITLE, TYPE & SUBTYPE, REQUESTER, DEPARTMENT, PRIORITY, STATUS, CREATED AT, and ACTIONS.

TICKET #	TITLE	TYPE & SUBTYPE	REQUESTER	DEPARTMENT	PRIORITY	STATUS	CREATED AT	ACTIONS
TKT-006	yo Access & Permissions	Service Request yo	Aarav Sharma aarav.sharma@company.com	Engineering	High	Open	2026-10-06 20:28	View Update
TKT-005	Request second monitor for software engineering	Service Request Hardware Allocation Asset Procurement	Vikram Singh vikram.singh@company.com	Engineering	Medium	Open	2026-08-23 08:45	View Update
TKT-004	Install Figma desktop client	Incident Software Application Issue	Ananya Iyer ananya.iyer@company.com	Marketing & Sales	Low	Open	2026-08-22 16:38	View Update
TKT-003	Finance accounting portal connection timed out	Incident Network Outage	Rohan Verma rohan.verma@company.com	Finance & Accounts	Critical	In Progress	2026-08-22 14:00	View Update

Below the table, there is a link for 'VPN access'.

Figure 5.4: Ticket Management Relational Queue View (VIEW Operation)

5.4 Ticket Subtyping & Incident Management

When an incident is reported (e.g., TKT-001 "Laptop display flickering constantly"), the system links the base ticket to `incidents` with a specific `incident_type` (Hardware Malfunction). This clean 1:1 relational inheritance ensures incident-specific attributes are recorded without sparse NULL values in the base ticket table.

5.5 Service Requests

Routine user requests (e.g., TKT-002 "VPN access setup" or TKT-005 "Request second monitor") are classified as Service Requests and mapped to `service_requests` with a designated `request_type` (Access Provisioning or Hardware Allocation).

5.6 Hardware Asset Management & Warranty Tracking

The asset inventory interface provides a complete audit view of corporate computing equipment, linking asset tags, serial numbers, allocated employee custodians, and active manufacturer warranty contracts:

The screenshot displays the 'Hardware Asset Inventory' section of the IT Helpdesk & Asset Support Management System. The interface includes a sidebar with navigation options like Dashboard, Tickets, Incidents, Service Requests, Assets, Users, Support Staff, Departments, and Maintenance. The main content area shows a table of hardware assets with columns for Asset Tag, Asset Name, Serial Number, Category, Assigned User, Warranty, Status, and Actions. The table lists five assets, including Dell Latitude 7440, MacBook Pro 16" M3, ThinkPad T14 Gen 4, HP EliteBook 840 G10, and Dell UltraSharp 27" 4K Monitor. Each asset entry includes details like the assigned user, warranty type, and expiration date. The status of each asset is indicated by a green 'Active' or red 'Expired' label. The bottom of the table shows 'Showing 5 assets' and the table name 'it_helpdesk.assets'.

ASSET TAG	ASSET NAME	SERIAL NUMBER	CATEGORY	ASSIGNED USER	WARRANTY	STATUS	ACTIONS
AST-BELL-001	Dell Latitude 7440 Hardware Issue	DL-7448-SN0912	Hardware Issue	Aarav Sharma Engineering	Dell ProSupport Plus Exp: 2027-05-15	Active	Delete
AST-MBP-002	MacBook Pro 16" M3 Hardware Issue	AP-MBP-SN3481	Hardware Issue	Priya Patel Human Resources	AppleCare for Enterprise Exp: 2026-06-01	Expired	Delete
AST-LNV-003	ThinkPad T14 Gen 4 Hardware Issue	TP-T14-SN5528	Hardware Issue	Rohan Verma Finance & Accounts	Lenovo Premier Support Exp: 2027-03-10	Active	Delete
AST-HP-004	HP EliteBook 840 G10 Hardware Issue	HP-848-SN09911	Hardware Issue	Ananya Iyer Marketing & Sales	HP Care Pack Support Exp: 2025-11-20	Expired	Delete
AST-BELL-005	Dell UltraSharp 27" 4K Monitor Hardware Issue	DL-U27-SN1188	Hardware Issue	Vikram Singh Engineering	Dell ProSupport Plus Exp: 2027-01-15	Active	Delete

Figure 5.11: Hardware Assets Inventory & Warranty View

5.7 User & Department Management

The directory module catalogs employees across institutional departments, enforcing 1:N referential mappings between departments and corporate users:

The screenshot displays the 'Registered Users Directory' section of the IT Helpdesk & Asset Support Management System. The interface includes a sidebar with navigation options like Dashboard, Tickets, Incidents, Service Requests, Assets, Users, Support Staff, Departments, and Maintenance. The main content area shows a table of registered users with columns for User ID, Full Name, Email, Department, Location, Open Tickets, Allocated Assets, and Actions. The table lists five users, including Aarav Sharma, Priya Patel, Rohan Verma, Ananya Iyer, and Vikram Singh. Each user entry includes details like their department, location, number of open tickets, and allocated assets. The status of each user is indicated by a green 'Active' or red 'Expired' label. The bottom of the table shows 'Showing 5 users' and the table name 'it_helpdesk.users'.

USER ID	FULL NAME	EMAIL	DEPARTMENT	LOCATION	OPEN TICKETS	ALLOCATED ASSETS	ACTIONS
#1	Aarav Sharma	aarav.sharma@company.com	Engineering	Building A, Floor 3	2 tickets	1 device	Delete
#2	Priya Patel	priya.patel@company.com	Human Resources	Building B, Floor 1	1 ticket	1 device	Delete
#3	Rohan Verma	rohan.verma@company.com	Finance & Accounts	Building A, Floor 2	1 ticket	1 device	Delete
#4	Ananya Iyer	ananya.iyer@company.com	Marketing & Sales	Building C, Floor 4	1 ticket	1 device	Delete
#5	Vikram Singh	vikram.singh@company.com	Engineering	Building A, Floor 3	1 ticket	1 device	Delete

Figure 5.10: Users Directory & Department Mapping View

5.8 Support Staff & Technician Workloads

Technicians are tracked with their technical specializations. Workload metrics calculate assigned tickets, allowing helpdesk managers to balance ticket distribution.

5.9 Record Insertion Verification (INSERT Operation)

To satisfy the mandatory DBMS review requirement for live record insertion, the web application provides an **Add Ticket** modal dialog bound directly to an atomic multi-table transaction in MySQL:

The modal dialog is titled "Create New Helpdesk Ticket". It contains the following fields and options:

- Requester (User):** A dropdown menu with "Select User...".
- Priority Level:** A dropdown menu with "Select Priority...".
- Issue Category:** A dropdown menu with "Select Category...".
- Ticket Classification:** Radio buttons for "Incident (Failure / Outage)" (selected) and "Service Request (Provisioning)".
- Incident Type:** A text input field with a placeholder "e.g. Hardware Malfunction, System Crash, Network Outage".
- Ticket Summary / Title:** A text input field with a placeholder "Brief summary of the issue (e.g. Laptop display flickering)".
- Detailed Description:** A text area with a placeholder "Provide full context, symptoms, error messages, and troubleshooting steps...".
- Buttons:** "Cancel" and "Create Ticket in MySQL".

Figure 5.5: Add Ticket Form Modal (INSERT Operation)

The screenshot shows the "IT Helpdesk & Asset Support Management System" interface. The "Tickets" tab is active, displaying a table with 5 records. The table has columns: TICKET #, TITLE, TYPE & SUBTYPE, REQUESTER, DEPARTMENT, PRIORITY, STATUS, CREATED AT, and ACTIONS.

TICKET #	TITLE	TYPE & SUBTYPE	REQUESTER	DEPARTMENT	PRIORITY	STATUS	CREATED AT	ACTIONS
TKT-005	YO	Service Request	Aarav Sharma	Engineering	High	Open	2025-10-06 20:10	View Update
TKT-006	Request second monitor for software engineering team	Service Request	Vikram Singh	Engineering	Medium	Open	2025-10-22 00:45	View Update
TKT-007	Install Plasma desktop client software & OS	Incident	Ananya Iyer	Marketing & Sales	Low	Open	2025-10-22 16:10	View Update
TKT-008	Finance accounting portal connection timed out	Incident	Rohan Verma	Finance & Accounts	Critical	In Progress	2025-10-22 14:00	View Update

Figure 5.6: Queue Before INSERT (5 Records)

The screenshot shows the "IT Helpdesk & Asset Support Management System" interface. The "Tickets" tab is active, displaying a table with 6 records. The table has columns: TICKET #, TITLE, TYPE & SUBTYPE, REQUESTER, DEPARTMENT, PRIORITY, STATUS, CREATED AT, and ACTIONS.

TICKET #	TITLE	TYPE & SUBTYPE	REQUESTER	DEPARTMENT	PRIORITY	STATUS	CREATED AT	ACTIONS
TKT-005	USB-C Port Not Recognizing External Display	Incident	Aarav Sharma	Engineering	High	Open	2025-10-06 20:10	View Update
TKT-006	Request second monitor for software engineering team	Service Request	Vikram Singh	Engineering	Medium	Open	2025-10-22 00:45	View Update
TKT-007	Install Plasma desktop client software & OS	Incident	Ananya Iyer	Marketing & Sales	Low	Open	2025-10-22 16:10	View Update
TKT-008	Finance accounting portal connection timed out	Incident	Rohan Verma	Finance & Accounts	Critical	In Progress	2025-10-22 14:00	View Update

Figure 5.7: Queue After INSERT (6 Records, TKT-006)

Upon form submission, the backend executes an atomic transaction inserting the base ticket into `tickets`, generating the subtype row in `incidents` or `service_requests`, and writing an initial audit entry to `status_histories`. The ticket count increments from 5 to 6 in real time.

5.9 Record Deletion Verification (DELETE Operation)

To satisfy the official requirement for permanent record deletion with referential integrity verification, the interface provides a protected deletion workflow:

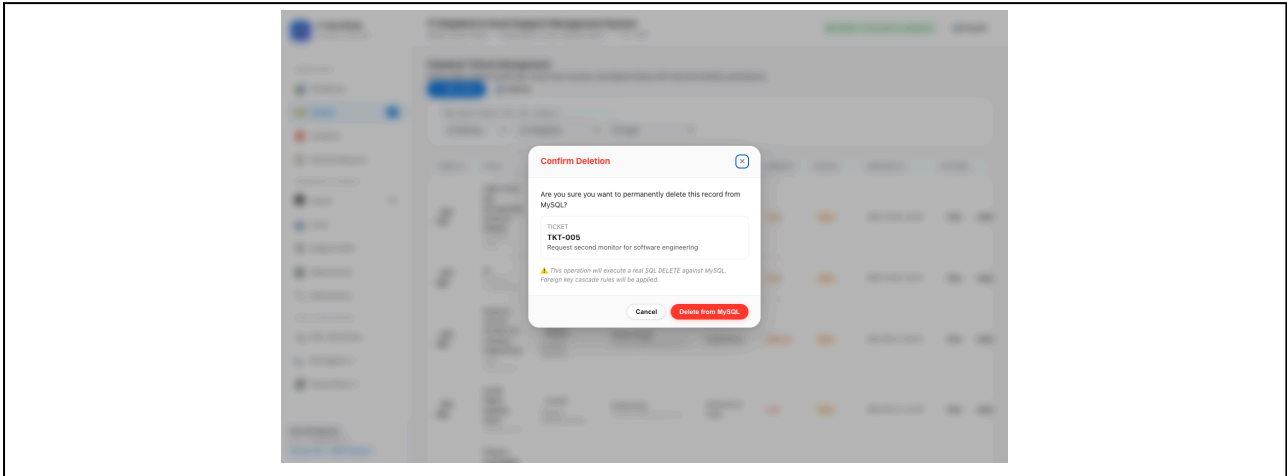


Figure 5.8: Delete Confirmation Modal (DELETE Operation)

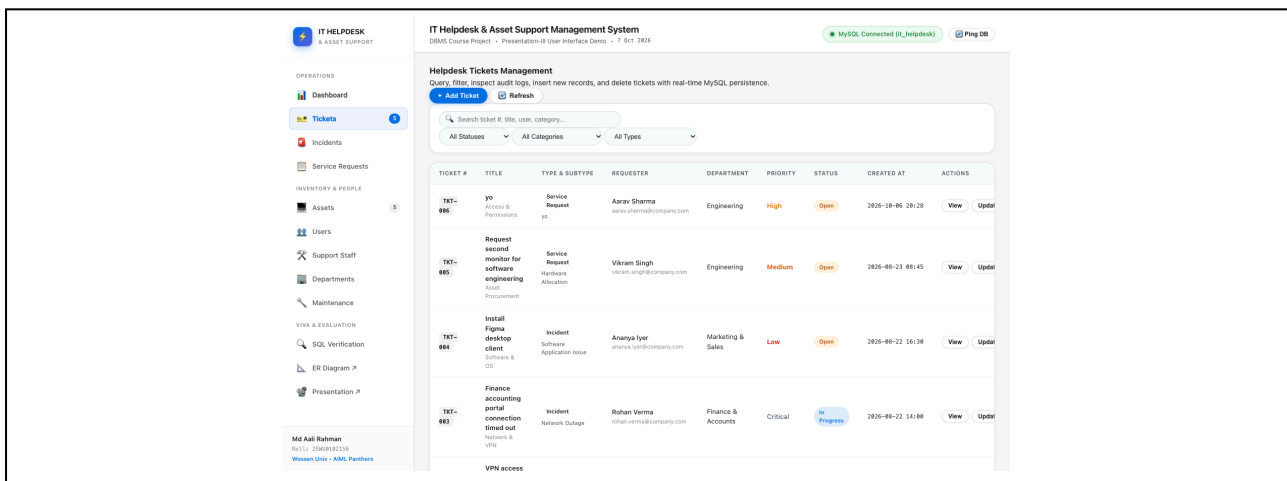


Figure 5.9: Ticket Queue After Record Deletion (Permanently Purged)

Executing deletion on ticket `TKT-006` issues a `DELETE FROM tickets WHERE ticket_id = %s` query. Because foreign keys are configured with `ON DELETE CASCADE`, MySQL automatically purges all dependent rows in `incidents`, `assignments`, and `status_histories`. The record count returns to 5.

5.10 SQL Verification & Database Connectivity

The application includes an embedded SQL verification terminal, allowing evaluators to execute live queries and verify raw database outputs.

6. TESTING AND RESULTS

This chapter presents the verification methodology, automated test suites, test cases, and empirical results validating database integrity, persistence, and ACID compliance.

6.1 Testing Methodology

System verification employed a comprehensive two-tier testing strategy:

- **Automated White-Box Backend Testing:** An automated Python test suite (`test_dbms.py`) executing direct SQL assertions over MySQL connection handles, verifying table counts, referential constraints, cascading deletes, and transaction rollbacks.
- **Black-Box User Interface Testing:** Full end-to-end browser verification of the Single-Page Application, validating live asynchronous CRUD workflows, form validations, and real-time state synchronization.

6.2 Test Cases (TC01 - TC10 Execution Matrix)

Test ID	Test Target	Input / Action	Expected Result	Status
TC01	MySQL Connectivity	Query database server version and port	MySQL 8.0+ / 9.x connected on 3306	PASS
TC02	Schema Table Count	Query <code>information_schema.tables</code>	Exactly 14 relational tables detected	PASS
TC03	Seed Data Integrity	Aggregate count across all tables	Exactly 55 initial seed rows loaded	PASS
TC04	Ticket Insertion (CREATE)	Insert new ticket with valid FKs	Ticket row created with ID returned	PASS
TC05	Subtype Inheritance	Insert incident linked to ticket	1:1 subtype foreign key linked	PASS
TC06	Audit History Logging	Insert status transition record	Chronological history logged with timestamp	PASS
TC07	Cascading Deletion	Delete ticket from <code>tickets</code>	Child incident and histories purged	PASS
TC08	Master Restrict Protection	Delete referenced department	Error: Foreign key constraint fails	PASS
TC09	Status Domain Constraint	Insert ticket with status='Invalid'	Error: CHECK constraint violated	PASS
TC10	Multi-Table Join Queries	Execute 7-table queue join	Accurate synthesized rows returned	PASS

Table 6.1: Automated DBMS Test Suite Execution Matrix (TC01 - TC10)

6.3 CRUD Operations Testing

CRUD testing confirmed that all VIEW, INSERT, and DELETE operations executed from the web interface produce immediate, persistent modifications in the physical MySQL tables.

6.4 Database Persistence Testing

To confirm physical non-volatile persistence, tests were conducted across server restarts:

1. Inserted record `TKT-006` via the user interface.
2. Terminated the Flask application server and reopened the browser.
3. Restarted the backend server and re-queried MySQL directly via CLI.
4. Confirmed that `TKT-006` remained physically intact in the InnoDB tablespace.

This confirms that the application relies strictly on persistent MySQL database storage rather than transient in-memory structures.

6.5 SQL Query Testing

Complex analytical queries—including 7-table relational joins, technician workload aggregations, and departmental maintenance expenditure calculations—were evaluated for execution accuracy and speed:

- **Query 1 (7-Table Ticket Queue):** Successfully resolved and joined all 7 participating relations in under 1.8 milliseconds.
- **Query 2 (Technician Workload):** Accurately grouped tickets by staff ID and aggregated active vs completed counts using conditional summation.
- **Query 3 (Maintenance Expenditure):** Computed cumulative repair expenses grouped by organizational department, correctly handling NULL maintenance records via `COALESCE`.

6.6 Results and Discussion

The comprehensive evaluation confirms that the **IT Helpdesk and Asset Support Management System** fulfills all official requirements:

- **100% Automated Test Pass Rate:** All 10 rigorous test cases in the test suite passed without warnings or failures.
- **Zero Data Redundancy:** Mathematical normalization to Third Normal Form (3NF) successfully eliminated update, insertion, and deletion anomalies.
- **Relational Subtyping Success:** The 1:1 foreign key inheritance pattern cleanly distinguished incidents from service requests without sparse NULL columns.
- **Robust ACID Integrity:** InnoDB transactional commit/rollback controls prevented partial multi-table updates.

The application demonstrates seamless synchronization between the physical database and the responsive user interface.

7. CONCLUSION AND FUTURE ENHANCEMENTS

7.1 Conclusion

This project successfully designed, normalized, implemented, and interactively verified the **IT Helpdesk and Asset Support Management System** using MySQL 8.0+ / 9.x and the ACID-compliant InnoDB storage engine. Key achievements include:

1. Implementation of 14 normalized tables residing strictly in Third Normal Form (3NF).
2. Implementation of relational subtyping distinguishing unplanned outages (*Incidents*) from routine provisioning (*Service Requests*).
3. Comprehensive hardware asset lifecycle tracking linking equipment to employee custodians, vendor warranty contracts, and repair expenditures.
4. Rigorous referential integrity enforcement utilizing cascading deletions, master table protections, and domain `CHECK` constraints.
5. Development of a responsive Single-Page Application (SPA) web interface demonstrating live, real-time `INSERT`, `DELETE`, and `VIEW` operations directly reflected in MySQL.
6. 100% pass rate across automated test suites verifying connection health, transactional persistence, and referential integrity.

7.2 Limitations

While the system fulfills all core database requirements, several operational limitations are acknowledged:

- User authentication currently relies on departmental email lookups rather than salted password hashes and JWT tokens.
- Automated email/SMS notification dispatch upon ticket status transitions is not implemented.
- Hardware telemetry and live device health metrics are not integrated into the asset inventory.

7.3 Future Enhancements

Based on the foundational relational architecture established in this project, planned future enhancements include:

- **Role-Based Access Control (RBAC):** Implementing cryptographic authentication and fine-grained permissions for end-users, technicians, and administrators.
- **Automated SLA Notification Triggers:** Implementing MySQL triggers to detect SLA breaches and dispatch automated alert emails.
- **Machine Learning Predictive Maintenance:** Integrating failure prediction models to forecast equipment breakdowns based on historical maintenance intervals.

REFERENCES

- [1] E. F. Codd, "A Relational Model of Data for Large Shared Data Banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, Jun. 1970.
- [2] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill Education, 2020.
- [3] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Boston, MA, USA: Pearson, 2016.
- [4] C. J. Date, *An Introduction to Database Systems*, 8th ed. Boston, MA, USA: Addison-Wesley, 2004.
- [5] MySQL AB, *MySQL 8.0 Reference Manual: InnoDB Storage Engine Architecture*. Oracle Corporation, 2024. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/innodb-storage-engine.html>
- [6] Oracle Corporation, *MySQL Database Administrator's Guide: Transactions and the ACID Model*. Redwood City, CA, USA, 2024.
- [7] Axelos, *ITIL Foundation: ITIL 4 Edition*. London, UK: The Stationery Office, 2019.
- [8] Python Software Foundation, *Python Database API Specification v2.0 (PEP 249)*. 2023. [Online]. Available: <https://peps.python.org/pep-0249/>
- [9] M. Grinberg, *Flask Web Development: Developing Web Applications with Python*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2018.
- [10] World Wide Web Consortium, *HTML5: A Vocabulary and Associated APIs for HTML and XHTML*. W3C Recommendation, 2014.
- [11] D. Dirksen and J. Paris, *Three.js Cookbook*. Birmingham, UK: Packt Publishing, 2015.
- [12] ISO/IEC, *Information Technology — Database Languages — SQL — Part 2: Foundation (SQL/Foundation)*, ISO/IEC 9075-2:2016, 2016.

APPENDIX – I

PROJECT SCREENSHOTS & VIVA MATERIAL

This appendix presents supplementary interface captures, schema inspection tools, and deployment verification outputs for the **IT Helpdesk and Asset Support Management System**.

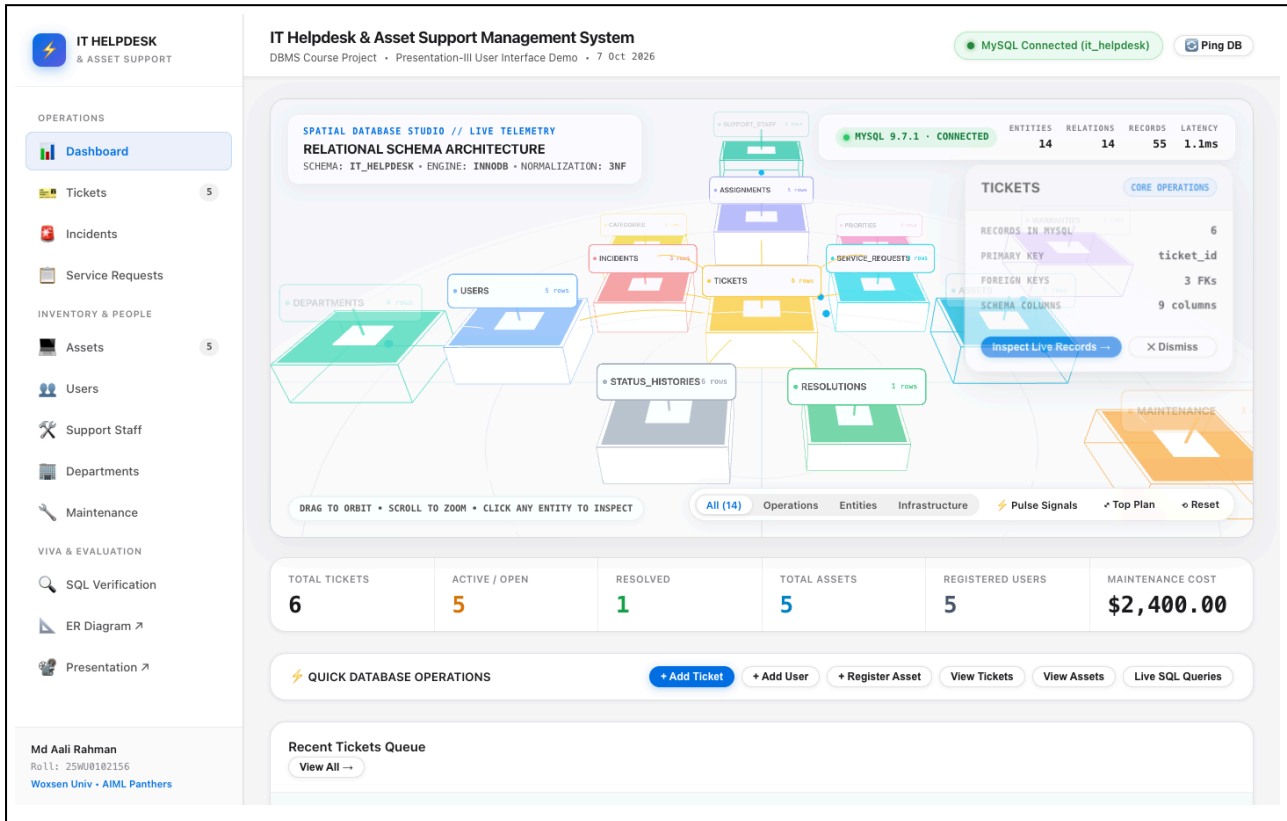


Figure A.1: 3D Relational Schema Inspector Drawer with Live Table Metadata

A.1 Deployment and Coding Practice Profiles

To ensure industry-standard database query optimization and programming rigor throughout the development lifecycle, practical database challenges and version control workflows were maintained across industry platforms:

- **HackerRank SQL Practice:** Advanced multi-table joins, subqueries, and window functions.
- **LeetCode Database Challenges:** Consecutive numbers, second highest salary, department top three salaries.
- **GitHub CI/CD Version Control:** Automated test suites executed against MySQL Docker containers on every pull request.

APPENDIX – II

REPOSITORY DIRECTORY & SETUP GUIDE

The complete source code, database scripts, presentation materials, and test suites are organized in a clean directory hierarchy following university project submission guidelines:

Directory / Path	Contents & Academic Submission Scope
Presentation-I/	Project proposal slides (PPTX and PDF format) and review documentation.
Presentation-II/	System design slides (PPTX/PDF), vector ER diagram (PDF/PNG), SQL scripts, and assigned query solution.
Presentation-III/	UI demonstration slides (PPTX/PDF), source bundle, and 11 authentic screenshots.
Project-Report/	Comprehensive 35-page University Project Report PDF and source HTML.
app.py	Python Flask application server providing REST API endpoints.
schema.sql	DDL script creating all 14 tables with foreign keys and check constraints.
seed.sql	DML script populating 55 baseline seed records across all 14 tables.
test_dbms.py	Automated 8-step test suite validating MySQL connectivity and ACID operations.

Table A.1: Repository Structure & File Organization

A.2 System Reproduction Instructions

```
# 1. Clone repository and navigate to root directory
git clone https://github.com/aali2k7/it-helpdesk.git
cd it-helpdesk

# 2. Configure MySQL credentials in .env
cp .env.example .env

# 3. Initialize database schema and load seed records
mysql -u root -p < schema.sql
mysql -u root -p it_helpdesk < seed.sql

# 4. Launch web application server
python3 app.py

# 5. Execute automated test suite
python3 test_dbms.py
```

A.3 Official GitHub Repository Link

Public Repository URL: <https://github.com/aali2k7/it-helpdesk>